

Por dentro do **React** e do React Native

A proposta de arquitetura já decidiu o quê: Expo, React Native, Expo Router, Supabase. Esta aula explica como essas peças funcionam por dentro — componente, estilo, layout, hook, Context API — e o que mais existe no ecossistema.

StyleSheet & Flexbox

layout nativo

Hooks nativos

useState ... useReducer

Context API

estado global

Hooks do ecossistema

navegação · dados · animação

0 que fica pronto até o fim do encontro

PARTE 1 · COMPONENTE, ESTILO E LAYOUT

- › Componente, JSX, props vs. state
- › React Native não é HTML: View, Text e os componentes nativos
- › StyleSheet e Flexbox em profundidade, com diagrama dos eixos

PARTE 2 · HOOKS, CONTEXT & ARQUITETURA

- › Hooks básicos (useState, useEffect) e avançados (useRef, useMemo, useCallback, useReducer)
- › Context API, custom hooks e catálogo de hooks do ecossistema RN
- › Expo Router, gateways de hardware e testes de hook

A stack aprovada, por camada

CAMADA	TECNOLOGIA	O QUE VAMOS EXPLICAR HOJE
UI	React Native + Expo Router	Componentes, JSX, Flexbox, navegação por arquivo
Estilo/layout	StyleSheet + Flexbox	Eixo principal x cruzado, propriedades mais usadas
Estado local	useState / useReducer	Rascunho de formulário, fluxo multi-etapas
Efeitos colaterais	useEffect / useRef	Listener de rede (NetInfo), foco de campo
Estado global	Context API	Sessão Supabase visível para todas as telas
Ponte c/ domínio	Custom Hooks	useAtividades, useAuth chamando use cases

Um componente é uma função que devolve interface

A IDEIA CENTRAL

- › Componente = função que recebe props e retorna JSX.
- › JSX é açúcar sintático para `React.createElement(...)`.
- › React chama a função de novo quando o dado muda: re-render.
- › Componentes se aninham em árvore — tela = componente pai.

```
• • • adapters/screens/LoginScreen.tsx

function LoginScreen() {
  // roda de novo a cada re-render
  return (
    <View style={styles.container}>
      <Text style={styles.title}>
        Entrar no App
      </Text>
      <EmailField />
      <SenhaField />
      <BotaoEntrar />
    </View>
  );
}
```

React Native ≠ HTML

React Native não roda num navegador. JSX descreve componentes nativos de verdade — iOS/Android renderizam a UI.

WEB (REACT)	MOBILE (REACT NATIVE)	DETALHE
<div>	<View>	Container puro, sem tags semânticas
<p> / 	<Text>	Todo texto precisa estar dentro de <Text>
	<Image>	Fonte via { uri } ou require()
CSS + classes	StyleSheet.create()	Objeto JS — próximos 3 slides
flex opcional	Flexbox por padrão	Todo View já é flex-direction: column
onClick	onPress	Toque, não clique de mouse

StyleSheet: como o React Native estiliza

COMO FUNCIONA

- › Sem CSS, sem cascata, sem seletores: cada componente recebe um objeto direto na prop style.
- › `StyleSheet.create({...})` valida as chaves e otimiza o objeto — continua sendo JavaScript puro.
- › Número sem unidade = density-independent pixel (dp); cada aparelho converte para pixel real.
- › style aceita um array — `style=[base, condicional && destaque]` — o último que bate ganha.

```
• • • adapters/screens/AtividadesFormScreen.styles.ts
```

```
const styles = StyleSheet.create({
  container: {
    flex: ,
    padding: ,
    backgroundColor: "#FFFFFF",
  },
  titulo: {
    fontSize: ,
    fontWeight: "600",
    marginBottom: ,
  },
  botaoDesativado: {
    opacity: .,
  },
});

<View style={[styles.container]}>
  <Text style={styles.titulo}>Bloco B</Text>
</View>
```

Flexbox: dois eixos, uma pegadinha

flexDirection: "row"

(padrão na web)

eixo principal (main axis) →

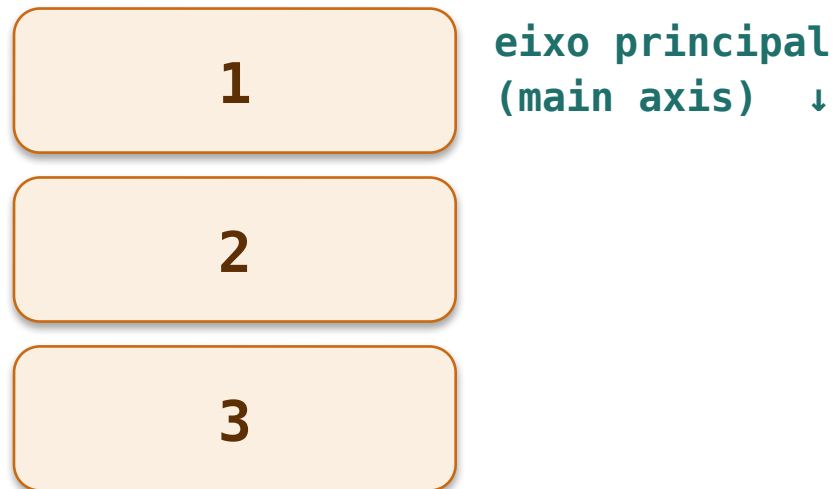


↕ eixo cruzado (cross axis)

flexDirection: "column"

(padrão no React Native!)

↔ eixo cruzado (cross axis)



Pegadinha nº 1 de quem vem da web: todo `<View>` em React Native já nasce `flexDirection: "column"`. Para imitar uma linha da web, é preciso escrever `flexDirection: "row"` explicitamente.

Flexbox na prática: propriedades mais usadas

PROPRIEDADE	VALORES COMUNS	EFEITO
flexDirection	row · column	Define o eixo principal (RN começa em column)
justifyContent	flex-start · center · space-between	Alinha os itens ao longo do eixo principal
alignItems	flex-start · center · stretch	Alinha os itens ao longo do eixo cruzado
flex	1 · 2 · 0	Fração do espaço livre que o item ocupa
flexWrap	wrap · nowrap	Quebra para a próxima linha/coluna quando falta espaço
gap	8 · 16 · 24	Espaço fixo entre itens, sem margin manual
alignSelf	auto · center · flex-end	Sobrescreve alignItems só para este item

Montando uma tela real com Flexbox

```
• • • adapters/screens/AtividadesFormScreen.tsx

<View style={{ flex: 1 }}>

  <View style={{ height: 40 }}>
    <CabecalhoTela titulo="Bloco B" />
  </View>

  <View style={{ flex: 1 }}>
    <ConteudoRolavel />
  </View>

  <View style={{
    flexDirection: "row",
    justifyContent: "space-between",
    padding: 10,
  }}>
    <BotaoVoltar />
    <BotaoEnviar />
  </View>

</View>
```

LENDO O LAYOUT

- › Container raiz flex: 1 ocupa toda a tela — cabeçalho, conteúdo e rodapé em coluna.
- › flex: 1 no meio: o conteúdo rolável estica e absorve todo o espaço sobrando entre cabeçalho e rodapé.
- › O rodapé vira flexDirection: row + justifyContent: space-between para colocar os botões nas pontas.

Props chegam de fora. State mora dentro.

PROPS

- › Parâmetros que o componente recebe do pai
- › Somente leitura — nunca alteradas por quem recebe
- › Ex.: `<Critério nota={8} label="Pontualidade" />`

STATE

- › Memória interna, entre re-renders
- › Muda só com a função setter — nunca por atribuição direta
- › Ex.: texto que o aluno digita em "Atividades desenvolvidas"

Regra prática (seção 03, Boundary-Control-Entity): a entidade de domínio (PeríodoAvaliação) não sabe o que é state React — quem guarda state é a tela, que entrega o valor final ao use case.

Hook = função que “encaixa” em recursos do React

POR QUE EXISTEM

- › Antes de 2019, só classes tinham state e ciclo de vida.
- › Hooks trazem esse poder para funções — sem class, sem this.
- › Todo hook começa com use: useState, useEffect, useRef, useContext, e os que a equipe criar (useAtividades).

AS DUAS REGRAS DOS HOOKS

- 1 Só no topo da função.** Nunca dentro de if, laço ou função aninhada — a ordem de chamada precisa ser sempre igual.
- 2 Só em componentes ou em outros hooks.** Nunca dentro de uma função comum ou de um use case do domínio.

Guardando o rascunho do Bloco B

```
• • • adapters/screens/AtividadesFormScreen.tsx

function AtividadesFormScreen() {
  const [descricao, setDescricao] =
    useState("");

  return (
    <TextInput
      value={descricao}
      onChangeText={setDescricao}
      placeholder="Descreva a atividade"
    />
  );
}
```

LENDO A CHAMADA

- › `useState("")` devolve um par: valor atual + setter.
- › `setDescricao(texto)` agenda um novo render — nunca muda a variável na hora.
- › Cada tela tem seu próprio state; nada é compartilhado automaticamente.

Ouvindo a conexão para acionar a fila (RNF05)

• • • adapters/hooks/useSyncStatus.ts

```
function useSyncStatus() {  
  const [online, setOnline] = useState(true);  
  
  useEffect(() => {  
    const unsubscribe =  
      NetInfo.addEventListener(  
        state => setOnline(state.isConnected)  
      );  
  
    return () => unsubscribe(); // limpeza  
  }, []); // roda só na montagem  
  
  return online;  
}
```

AS TRÊS PARTES DE UM EFFECT

- 1 Efeito.** Código que sai do React — assinar evento, ler sensor, chamar SyncGateway.
- 2 Dependências.** `[]` roda uma vez; com valores dentro, roda de novo quando eles mudam.
- 3 Limpeza.** O return desfaz o efeito — evita vaziar listeners de NetInfo.

Valor mutável que não pede re-render

```
• • • adapters/screens/BuscaRelatorioScreen.tsx
```

```
function BuscaRelatorioScreen() {  
  const inputRef = useRef(null);  
  
  useEffect(() => {  
    inputRef.current.focus();  
  }, []);  
  
  return (  
    <TextInput  
      ref={inputRef}  
      placeholder="Buscar relatório"  
    />  
  );  
}
```

PARA QUE SERVE

- › Guarda um valor entre renders sem disparar repintura ao mudar `.current`.
- › Referencia um elemento nativo de verdade: focar `TextInput`, rolar `FlatList`.
- › Também guarda "memória de bastidor" — id de `setInterval`, contador que a UI não precisa mostrar.

Evitando trabalho repetido a cada render

• • • adapters/screens/HistoricoRelatoriosScreen.tsx

```
const totalHoras = useMemo(  
  () => calcularCargaHoraria(periodos),  
  [periodos]  
);  
  
const renderItem = useCallback(  
  ({ item }) => <RelatorioCard  
    relatorio={item} />,  
  []  
);  
  
<FlatList data={periodos} renderItem={renderItem}  
/>
```

DUAS MEMORIZAÇÕES DIFERENTES

- › useMemo memoriza o resultado de um cálculo caro; só refaz quando a dependência muda.
- › useCallback memoriza a própria função — evita recriar uma referência nova em todo render.
- › Ganha peso em listas grandes (FlatList) e telas com muitos componentes-filho memoizados.

Estado complexo com ações explícitas

• • • adapters/screens/FormularioWizard.tsx

```
function reducer(estado, acao) {  
  switch (acao.type) {  
    case "AVANCAR":  
      return { ...estado, passo: estado.passo + 1 };  
    case "VOLTAR":  
      return { ...estado, passo: estado.passo - 1 };  
    default:  
      return estado;  
  }  
}  
  
const [estado, dispatch] = useReducer(reducer,  
  { passo: 1 });
```

QUANDO TROCAR O `useState`

- › O próximo estado depende do anterior e há várias transições possíveis.
- › Uma função `reducer(estado, ação)` decide o novo estado — mesmo padrão do Redux, sem lib extra.
- › Bom encaixe: formulário em passos (wizard) ou máquina de estado como `StatusPeriodo`.

Sem Context, a sessão vira prop em toda tela

PROP DRILLING

Aluno, Orientador e Coordenação logam pelo mesmo AuthGateway. Sem um lugar global, o usuário logado precisaria ser repassado manualmente em cada nível:

App → TabsLayout →

HistoricoRelatoriosScreen →

RelatorioCard → BadgeAutor

Quatro componentes recebendo usuario só para repassá-lo adiante — nenhum usa o valor diretamente.

A SAÍDA: CONTEXT API

- › Um Provider guarda o valor uma vez, no topo da árvore.
- › Qualquer descendente lê o valor com useContext — sem passar por ninguém no meio.
- › Ideal para dado que muitas telas precisam: sessão, tema, permissões.

Uma sessão, lida de qualquer tela

• • • adapters/context/AuthContext.tsx

```
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [usuario, setUsuario] = useState(null);

  useEffect(() => {
    authGateway.onSessionChange(setUsuario);
  }, []);

  return (
    <AuthContext.Provider value={{ usuario }}>
      {children}
    </AuthContext.Provider>
  );
}

// em qualquer tela descendente:
const { usuario } = useContext(AuthContext);
```

TRÊS PEÇAS DO PADRÃO

- 1** **createContext** cria o canal onde o valor vai viajar.
- 2** **<Provider value=...>** envolve a árvore uma vez, no App raiz.
- 3** **useContext(...)** qualquer tela lê o valor e re-renderiza sozinha quando ele muda.

Um hook próprio é a fronteira entre tela e use case

• • • adapters/hooks/useAtividades.ts

```
function useAtividades(periodoId) {  
  const [status, setStatus] =  
    useState("idle");  
  
  async function registrar(dados) {  
    setStatus("salvando");  
    await  
    registrarAtividadesUseCase.executar({  
      periodoId, ...dados,  
    });  
    setStatus("salvo");  
  }  
  
  return { status, registrar };  
}
```

POR QUE NÃO CHAMAR O USE CASE DIRETO?

- › A tela só sabe desenhar UI e reagir a toque.
- › O hook adapta o use case (puro, sem React) ao ciclo de vida do componente.
- › Proposta, seção 10: hooks adaptam o use case, não o substituem.

Hooks nativos do React – referência rápida

HOOK	PARA QUE SERVE	QUANDO USAR
useState	Guarda valor simples entre renders	Campo de formulário, contador, flag
useEffect	Roda efeito colateral após o render	Chamada de API, listener, timer
useLayoutEffect	Como useEffect, mas antes de pintar a tela	Medir e ajustar layout sem "piscar"
useRef	Guarda valor mutável sem re-render	Referência a input, id de timer
useMemo	Memoriza resultado de cálculo	Cálculo pesado, lista derivada grande
useCallback	Memoriza referência de função	Callback passado a componente memoizado
useReducer	Estado com transições explícitas	Fluxo multi-etapas, máquina de estado
useContext	Lê valor de um Provider	Sessão, tema, preferências globais

Hooks que aparecem em quase todo projeto React Native

BIBLIOTECA	HOOK	PARA QUE SERVE
React Navigation	useNavigation	Navegar sem receber props de navegação
React Navigation	useRoute	Ler parâmetros da rota atual
React Navigation	useFocusEffect	Rodar efeito quando a tela ganha foco
safe-area-context	useSafeAreaInsets	Respeitar notch/status bar em qualquer device
React Native core	useWindowDimensions	Reagir a rotação de tela e tamanhos
React Native core	useColorScheme	Detectar tema claro/escuro do sistema
TanStack Query	useQuery / useMutation	Buscar e cachear dados remotos sem Redux
Reanimated	useSharedValue / useAnimatedStyle	Animações rodando na UI thread

Nenhum é obrigatório na proposta do App Estágio — mas todos aparecem em algum projeto React Native de produção.

React e React Native só aparecem numa camada

FRAMEWORKS & DRIVERS

expo-sqlite · expo-camera · supabase-js

ADAPTERS — AQUI VIVEM REACT & RN

Screens · useAtividades · useAuth · AuthContext · Gateways Expo

APPLICATION

RegistrarAtividadesUseCase · GerarPdfUseCase

DOMAIN

PeriodoAvaliacao · CameraGateway (interface) — zero React

Nenhum arquivo em domain/ ou application/ importa react ou react-native. Hooks, Context e telas só existem na camada Adapters.

Da pasta de arquivos ao sensor do celular

EXPO ROUTER

- › Cada arquivo em app/ é uma rota — sem configurar navegação à mão.
- › app/login.tsx → /login;
app/(aluno)/atividades.tsx → aba do aluno.
- › É a Boundary de entrada da seção 06 (BCE) — onde o toque chega ao sistema.

PADRÃO GATEWAY PARA HARDWARE

- › CameraGatewayExpo embrulha expo-camera atrás da interface do domínio.
- › useAtividades chama o use case, que chama o gateway — nunca expo-camera direto.
- › Testável: um CameraGateway fake substitui a câmera real sem tocar hardware.

Testing Library entra na ponta da pirâmide

```
• • • __tests__/AtividadesFormScreen.test.tsx

test("salva atividade ao tocar em enviar", async () => {
  const useCaseFake = { executar: jest.fn() };

  const { getByText, getByPlaceholderText } = render(
    <AtividadesFormScreen useCase={useCaseFake} />
  );

  fireEvent.changeText(
    getByPlaceholderText("Descreva a atividade"),
    "Suporte ao setor de RH"
  );
  fireEvent.press(getByText("Enviar"));

  expect(useCaseFake.executar).toHaveBeenCalled();
});
```

O QUE MUDA

- › render() monta o componente de verdade; fireEvent simula toque e digitação.
- › O use case entra fake — o teste garante que a tela chama a peça certa.
- › Por isso poucos testes de componente: cobrem interação, não regra de negócio.

Dez termos para não confundir

StyleSheet.create

Objeto de estilos validado pelo React Native — sem cascata, sem seletores.

Hook

Função que começa com use e encaixa em recursos do React dentro de um componente.

useEffect

Hook para efeitos colaterais — rede, listener, timer, sensor.

useMemo / useCallback

Memorizam resultado de cálculo e referência de função.

Context / Provider

Canal que carrega um valor pelo topo da árvore, lido com useContext.

Flexbox (eixos)

Main axis segue flexDirection; cross axis é o eixo perpendicular a ele.

useState

Guarda um valor que sobrevive entre re-renders e dispara um novo render ao mudar.

useRef

Valor mutável entre renders que não dispara repintura ao mudar.

useReducer

Estado com transições explícitas via função reducer(estado, ação).

Custom Hook

Hook próprio que adapta um use case ao ciclo de vida da tela.